LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 – Inheritance



NAMA : Ahnaf Ariacheda

NIM : 10313240037

KELAS : PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Inheritance (Pewarisan)
    - Konsep: Hubungan hirarki *"is-a"* dari umum ke spesifik, di mana *subclass* (kelas anak) mewarisi atribut dan perilaku dari *superclass* (kelas induk).
    - Cara Kerja: Mencegah duplikasi kode. Properti umum didefinisikan sekali di kelas induk agar otomatis dimiliki kelas anak, sementara kelas anak tetap bisa menambah properti khusnya sendiri.
2. Keyword super
    - Konsep: Mekanisme untuk mereferensikan atau mengakses *member* (atribut, method, atau konstruktor) milik *superclass*.
    - Cara Kerja: Digunakan oleh kelas anak untuk memanggil konstruktor induk saat inisialisasi awal atau untuk mengakses kembali method asli induk yang tertutup oleh modifikasi di kelas anak.
3. Method Overloading
    - Konsep: Pembuatan beberapa method dengan nama yang sama dalam satu kelas yang sama, tetapi memiliki perbedaan jumlah atau tipe data parameter.
    - Cara Kerja: Sistem secara otomatis mendeteksi bentuk input yang dimasukkan, lalu mengeksekusi versi method yang parameternya paling cocok dengan input tersebut.
4. Method Overriding
    - Konsep: Penulisan ulang method oleh *subclass* dengan nama, tipe data, dan parameter yang identik dengan method di *superclass*-nya.
    - Cara Kerja: Saat method dipanggil melalui objek kelas anak, program akan mengabaikan versi kelas induk dan memprioritaskan eksekusi versi milik kelas anak.

B. Guided

1. Package guided1 :

- **Class A.java**

```
package com.naocheda.guided.guided1;

public class A {
    int x;
    int y;

    void tampilXY(){
        System.out.println("Nilai x = " + x + ", y = " + y);
    }
}
```

- **Class B.java**

```
package com.naocheda.guided.guided1;

public class B extends A {
    int z;
    int hasil;

    void jumlahXYZ(){
        hasil = x + y +z;

        System.out.println("jumlah (x+y+z) = " + hasil);
    }
}
```

- **Class Demo.java**

```
package com.naocheda.guided.guided1;

public class Demo {
    public static void main(String[] args) {

        A superclass = new A();
        B subclass = new B();
```

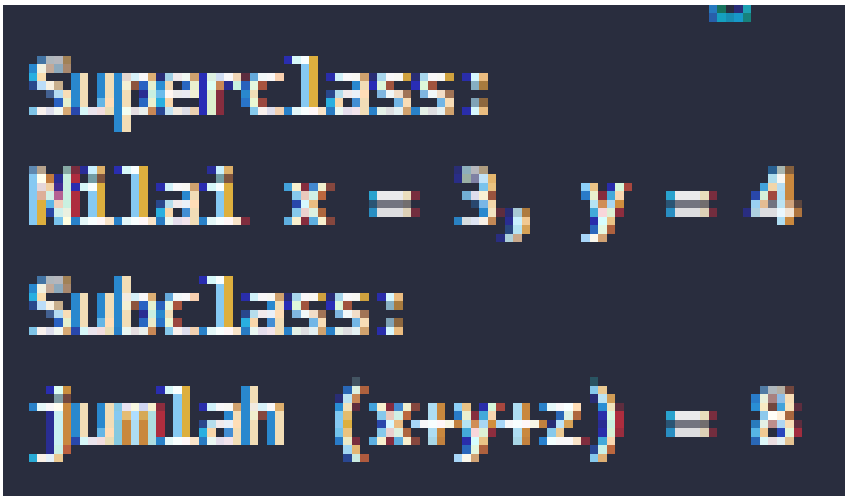
```
        superclass.x = 3;
        superclass.y = 4;

        System.out.println("Superclass: ");
        superclass.tampilXY();

        subclass.x = 1;
        subclass.y = 2;
        subclass.z = 5;

        System.out.println("Subclass: ");
        subclass.jumlahXYZ();
    }
}
```

Output :



2. **Package guided2 :**

- **Class Employee**

```
package com.naocheda.guided.guided2;


public class Employee {
    private String name;
    String departemen;

    public Employee(String name){
        this.name = name;
    }

    public void tampilNama(){
        System.out.println("nama: "+ name);
    }
}
```

- **Class Manager.java**

```
package com.naocheda.guided.guided2;

public class Manager extends Employee {
    private String alamat;

    public Manager(String nama, String alamat){
        super(nama);
        this.alamat = alamat;
    }

    public void tampilkanInformasi(){
        super.tampilNama();
        System.out.println("alamat: "+  alamat);
```

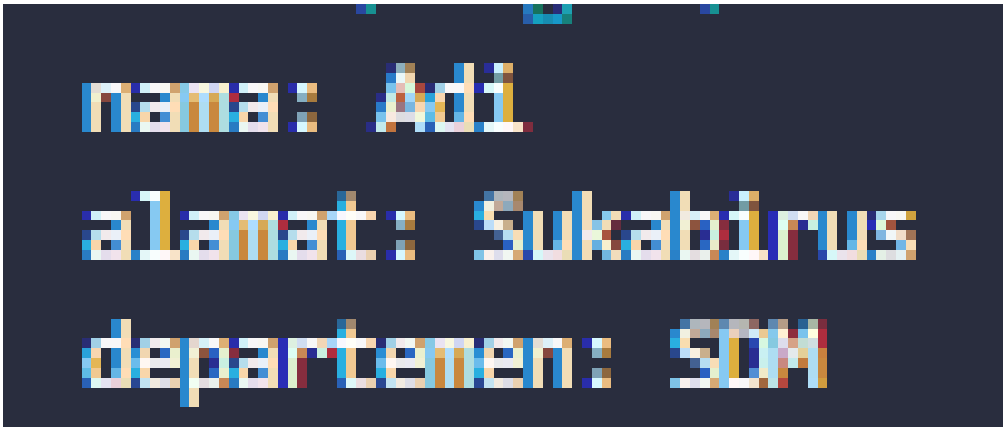
```
        System.out.println("departemen: "+ super.departemen);
    }
}
```

- **Class Demo.java**

```
package com.naocheda.guided.guided2;

public class Demo {
    public static void main(String[] args) {
        Manager adi = new Manager("Adi", "Sukabirus");
        adi.departemen = "SDM";
        adi.tampilkanInformasi();
    }
}
```

Output :



3. **Package guided3 :**

- **Class Lagu.java**

```java
package com.naocheda.guided.guided3;

public class Lagu {
    String pencipta;
    String judul;

    void isiParam(String param1){
        judul = param1;
        pencipta = "tidak dikenal";
    }

    void isiParam(String param1, String param2){
        judul = param1;
        pencipta = param2;
    }

    void tampilkanLagu(){
        System.out.println("Judul lagu: " + judul + ", pencipta: " + pencipta);
    }
}
```

- **Class Main.java**

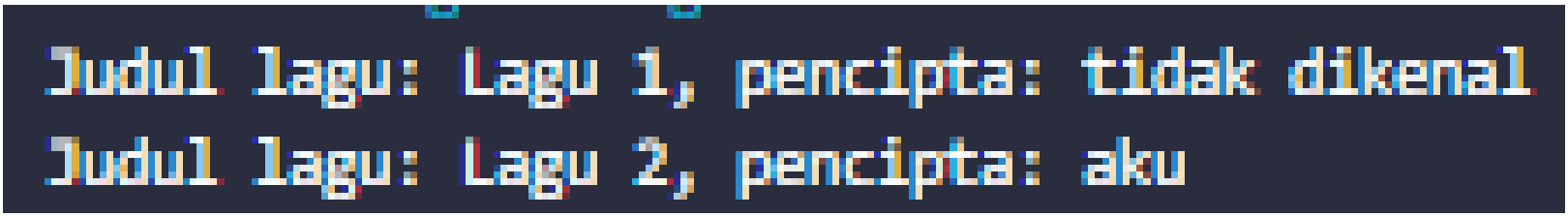
```java
package com.naocheda.guided.guided3;

public class Main {
    public static void main(String[] args) {
        Lagu d,e;
        d = new Lagu();
        e = new Lagu();

        d.isiParam("Lagu 1");
        e.isiParam("Lagu 2", "aku");

        d.tampilkanLagu();
        e.tampilkanLagu();
    }
}
```

Output :


Judul lagu: Lagu 1, pencipta: tidak dikenal
Judul lagu: Lagu 2, pencipta: aku

4. **Package guided4 :**

- **Class A.java**

```java
package com.naocheda.guided.guided4;

public class A {
    public void tampilkanKeLayar() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

- **Class B.java**

```java
package com.naocheda.guided.guided4;

public class B extends A {
    public void tampilkanKeLayar() {
        super.tampilkanKeLayar(); // Method milik superclas dipanggil
        System.out.println("Method milik class B dipanggil...");
    }
}
```

- **Main.java**

```java
package com.naocheda.guided.guided4;

public class Main {
    public static void main(String[] args) {
        B subOb = new B();
        subOb.tampilkanKeLayar();
    }
}
```

Output :

```
Method milik class A dipanggil...
Method milik class B dipanggil...
```

C. Unguided

1. Soal Unguided — Modul 5: Pemrograman Berorientasi Objek

```
# Soal Unguided – Modul 6 Inheritance

Sebuah tim *machine learning* ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis *deep learning* yang merupakan pengembangan dari model klasifikasi biasa.

Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

### Ketentuan

**1. Kelas induk (model umum)**
- Menyimpan **nama model** (teks) dan **jumlah data latih** (bilangan bulat).
- Memiliki *constructor* yang menerima kedua nilai tersebut, serta cara bagi kelas lain untuk membaca jumlah data latih.
- Memiliki method untuk menampilkan informasi.
- Memiliki dua method untuk menghitung biaya komputasi dengan nama yang sama (*overloading*):
  - Tanpa parameter: biaya = jumlah data latih × 2.
  - Dengan satu parameter bilangan bulat / jumlah epoch: biaya = jumlah data latih × epoch.

**2. Kelas turunan pertama (model regresi)**
- Mewarisi kelas induk dan tambahkan atribut **jumlah fitur**.
- *Constructor* menerima nama, jumlah data latih, dan jumlah fitur; gunakan *constructor* kelas induk.
- **Override** method: tampilkan informasi dari kelas induk.
- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) + (jumlah fitur × 10).
- Tambahkan **satu method baru** dengan nama yang sama dengan method penghitung biaya, tetapi menerima **dua parameter bilangan bulat** (jumlah *epoch* dan ukuran *batch*): biaya = jumlah data latih × epoch ÷ ukuran batch.
  Sehingga objek kelas ini memiliki **tiga versi** method penghitung biaya.

**3. Kelas turunan kedua (model klasifikasi)**
- Mewarisi kelas induk dan menambahkan atribut **jumlah kelas**.
- *Constructor* menerima nama, jumlah data latih, dan jumlah kelas; gunakan *constructor* kelas induk.
- **Override** method: tampilkan informasi dari kelas induk.
```

- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) × jumlah kelas.
- Versi penghitung biaya dengan parameter *epoch* **tidak** di-override.

4. Kelas turunan ketiga (model klasifikasi *deep learning*)
- Mewarisi **kelas turunan kedua** (bukan kelas induk langsung) dan menambahkan atribut **jumlah layer**.
- *Constructor* menerima nama, jumlah data latih, jumlah kelas, dan jumlah layer; gunakan *constructor* kelas di atasnya.
- **Override** method: tampilkan informasi dari kelas di atasnya.
- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas di atasnya) + (jumlah layer × 200).

5. Class Main
- Buat tiga objek berikut:

Jenis model	Nama	Data latih	Atribut tambahan
Regresi	`"Regresi Linier"`	500	4 fitur
Klasifikasi	`"Decision Tree"`	800	3 kelas
Klasifikasi *deep learning*	`"Neural Network"`	1000	5 kelas, 3 layer

- Simpan ketiga objek tersebut dalam satu array bertipe kelas induk.
- Gunakan perulangan untuk setiap elemen array: tampilkan informasi model, tampilkan biaya komputasi, lalu jumlahkan seluruh biaya.
- Setelah perulangan, tampilkan total biaya.
- Terakhir, tampilkan juga:
  - biaya 10 *epoch* untuk model `"Neural Network"` (ambil dari array),
  - biaya 10 *epoch* untuk model `"Regresi Linier"`,
  - biaya 10 *epoch* dengan ukuran *batch* 50 untuk model `"Regresi Linier"`.

Contoh Output

```
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
```

```
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

Jawaban :

- **Class MachineLearning.java**

```java
package com.naocheda.unguided;


public class MachineLearning {

    private String namaModel;

    private int jumlahDataLatih;


    public MachineLearning(String namaModel, int jumlahDataLatih) {

        this.namaModel = namaModel;

        this.jumlahDataLatih = jumlahDataLatih;

    }


    public int getJumlahDataLatih() {

        return jumlahDataLatih;
```

```
    }

    public void tampilkanInfo() {
        System.out.println("Nama Model: " + namaModel);
        System.out.println("Jumlah Data Latih: " + jumlahDataLatih);
    }

    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

- **Class Regresi.java**

```
package com.naocheda.unguided;

public class Regresi extends MachineLearning {
    private int jumlahFitur;

    public Regresi(String namaModel, int jumlahDataLatih, int jumlahFitur) {
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
```

```
        System.out.println("Jumlah Fitur: " + jumlahFitur);
    }

    @Override
    public int hitungBiaya(){
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    public int hitungBiaya(int epoch, int batchSize){
        return (getJumlahDataLatih() * epoch ) / batchSize;
    }
}
```

- **Class Klasifikasi.java**

```
package com.naocheda.unguided;

public class Klasifikasi extends MachineLearning {
    private int jumlahKelas;

    public Klasifikasi(String namaModel, int jumlahDataLatih, int jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah Kelas: " + jumlahKelas);
    }

    @Override
    public int hitungBiaya(){
        return super.hitungBiaya() * jumlahKelas;
    }
}
```

- **Class DeepLearning.java**

```java
package com.naocheda.unguided;

public class DeepLearning extends Klasifikasi {
    private int jumlahLayer;

    public DeepLearning(String namaModel, int jumlahDataLatih, int jumlahKelas, int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah Layer: " + jumlahLayer);
    }

    @Override
    public int hitungBiaya(){
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

- **Class Main.java**

```java
package com.naocheda.unguided;

public class Main {
    public static void main(String[] args) {
        MachineLearning[] models = new MachineLearning[3];
        models[0] = new Regresi("Regresi Linear", 500, 4);
        models[1] = new Klasifikasi("Decision Tree", 800, 3);
        models[2] = new DeepLearning("Neural Network", 1000, 5, 3);

        int totalBiaya = 0;
        for (MachineLearning m : models) {
```

```
            m.tampilkanInfo();
            int biaya = m.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");
            totalBiaya += biaya;
        }
        System.out.println("Total Biaya: " + totalBiaya);

        System.out.println("Biaya 10 epoch (Neural Network): " + models[2].hitungBiaya(10));
        System.out.println("Biaya 10 epoch (Regresi Linier): " + models[0].hitungBiaya(10));

        Regresi regresi = (Regresi) models[0];
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): " + regresi.hitungBiaya(10, 50));
    }
}
```

Output :

```
Nama Model: Regresi Linear
Jumlah Data Latih: 500
Jumlah Fitur: 4
Biaya komputasi: 1040
---
```

```
Nama Model: Decision Tree
Jumlah Data Latih: 800
Jumlah Kelas: 3
Biaya komputasi: 4800
---
```

```
Nama Model: Neural Network
Jumlah Data Latih: 1000
Jumlah Kelas: 5
Jumlah Layer: 3
Biaya komputasi: 10600
---
```

```
Total Biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

Penjelasan :

1. Class MachineLearning (Superclass)

- Menjadi kelas induk utama buat semua model. Atributnya diset private biar datanya aman (enkapsulasi), jadi butuh *method getter* supaya kelas anaknya bisa baca data latih.
- Punya dua versi *method* hitungBiaya() lewat konsep **Overloading** (satu tanpa parameter, satu lagi pakai parameter epoch).

2. Class Regresi (Subclass)

- Turunan langsung dari induk, dengan tambahan atribut jumlahFitur. Pakai super() di *constructor* buat ngelempar inisialisasi data ke atas.
- *Method* hitungBiaya() di-**Override** (ditimpa) untuk nambahin hitungan biaya fitur.
- Kelas ini juga bikin versi **Overloading** yang ketiga dan eksklusif, yaitu hitungBiaya dengan parameter epoch dan batchSize.

3. Class Klasifikasi (Subclass)

- Sama-sama turunan langsung, tapi dengan tambahan atribut jumlahKelas.
- Rumus hitung biayanya di-**Override** dengan cara dikali jumlah kelas.
- Menariknya, kelas ini sengaja nggak meng- *override* rumus yang pakai parameter epoch, jadi murni minjem perhitungan asli dari induknya.

4. Class DeepLearning (Subclass Bertingkat)

- Ini contoh *multi-level inheritance* karena dia nurun dari kelas Klasifikasi, bukan dari induk utama. Pemanggilan super()-nya jadi berantai ke atas.
- Untuk hitung biayanya, dia ngambil hasil dari bapaknya (Klasifikasi) lalu di-**Override** lagi dengan tambahan biaya *layer*.

5. Class Main (Eksekusi & Polymorphism)

- Tempat pembuktian **Polymorphism**. Kita bisa nyimpen objek Regresi, Klasifikasi, dan DeepLearning sekaligus di dalam satu *array* bertipe induk (MachineLearning[]).
- Pas di- *looping*, Java secara pintar (*dynamic dispatch*) ngejalani *method* yang pas buat masing-masing objek tanpa tertukar.
- Di sini juga dipakai teknik **Downcasting**, yaitu "maksa" tipe objek kembali jadi Regresi supaya *method* eksklusif hitungBiaya(epoch, batchSize) bisa dipanggil tanpa *error*.

Kesimpulan

Kesimpulannya, praktikum Modul 6 ini ngebuktiin kalau *Inheritance* (pewarisan) ngebantu banget buat bikin kode jadi lebih rapi dan bebas dari duplikasi. Kita cukup bikin satu *blueprint* di kelas induk (MachineLearning), lalu sifatnya diwarisin ke kelas anak-anaknya dengan bantuan *keyword* super.

Terus, berkat adanya *Polymorphism*, kita bisa mengelola berbagai macam kelas anak secara dinamis dalam satu wadah *array* yang sama. Kombinasi *Overloading* (variasi parameter) dan *Overriding* (modifikasi *method* warisan) bikin program OOP kita jadi jauh lebih fleksibel buat dikembangkan, tanpa harus nulis ulang kodenya dari nol.